# Weeks 7–8 Final Project: The Honest Backtest

Build, evaluate, and report on your own RL trading system

## The Brief

Over the past six weeks you built every piece of an RL trading system: a policy-gradient agent, a PPO training loop, three generations of trading environments, an evaluation harness with baselines, a controlled experiment on a market you designed, and an honest train/test study on real data. The final project asks you to put all of it together *your way*:

> **Design an improved trading environment, train a PPO agent on real historical data from at least two markets, evaluate it with the full discipline you have learned, and write a short report on what you found — whatever you found.**

This is not a treasure hunt for a profitable strategy. On real daily data, with your tools, the most likely honest outcome is "my improvements did not produce a reliable edge" — and a project that demonstrates that carefully, with clean methodology and a clear report, is a *complete success*. You are being assessed on process, judgement, and honesty, not on P&L.

There are no code skeletons this time. Everything you need is code you have already written; this document tells you what to build, not how.

## What You Already Have

Take stock before writing anything new. Your project folder already contains:

- **Data pipeline** (`week6_get_data.py`): download, clean, log returns, summary statistics.
- **Environment template** (`week6_historical_env.py`): replays real return windows through the Gymnasium interface.
- **Training** (any of your PPO scripts): the `PPO(...)` call you have now used on four different environments.
- **Evaluation harness** (`week4_evaluate.py`): `average_cumulative_pnl` and `plot_pnl`, with random and buy-and-hold baselines.
- **Method** (Weeks 5–6): verify the data before blaming the algorithm; chronological splits; multiple runs before conclusions; suspicion of profit.

The project is roughly 20% new code and 80% assembling, running, and interpreting.

## Before You Start: Three Small Tools

This is the only new material in the final project — two evaluation metrics and one convenience function. Read this section, then you are on your own.

### Sharpe Ratio

Final P&L hides *how bumpy the ride was*. Two strategies can both end at +20: one climbing steadily, one swinging between +80 and −40 on the way. The **Sharpe ratio** measures reward per unit of risk taken — the mean of the per-step rewards divided by their standard deviation, scaled to a yearly figure:

```
import numpy as np

def sharpe_ratio(step_rewards, periods_per_year=252):
    """Annualised Sharpe ratio of a sequence of per-step rewards.
    252 = trading days in a year (daily data)."""
    r = np.asarray(step_rewards, dtype=float)
    if r.std() == 0:
        return 0.0
    return float(r.mean() / r.std() * np.sqrt(periods_per_year))
```

Rules of thumb for daily data: negative = losing money; 0 to 0.5 = weak, likely noise; 0.5 to 1.0 = interesting if it survives multiple seeds and markets; above 1.0 = be suspicious first (Week 6 rules apply). Expect your agents to live in the first two bands.

### Maximum Drawdown

The **maximum drawdown** is the largest peak-to-trough fall of the cumulative P&L curve — the worst losing stretch an investor following the strategy would have sat through:

```
def max_drawdown(step_rewards):
    """Largest peak-to-trough decline of the cumulative P&L curve,
    in the same units as the reward."""
    equity = np.cumsum(np.asarray(step_rewards, dtype=float))
    peak = np.maximum.accumulate(equity)
    return float((peak - equity).max())
```

Always report it next to final P&L: "ended at +15 with a max drawdown of 60" tells a very different story from the +15 alone.

### Saving and Loading Models

Training takes minutes; you will not want to repeat it for every evaluation idea:

```
model.save("models/ppo_final")          # after training
model = PPO.load("models/ppo_final")    # later, for evaluation
```

That is all the new material. Everything below is requirements.

## Project Requirements

### 1. Data: Two Markets

Choose **at least two** instruments with roughly 8+ years of daily data (∼2 000 trading days) each. At most one may be a ticker you have already used (e.g. SPY); pick at least one *new* market you are genuinely curious about — an Indian index or stock (`^NSEI`, `RELIANCE.NS`), another country's index, a commodity ETF, whatever you like. For each market, report the Week 6 summary statistics (mean, std, lag-1 autocorrelation) in your report before any RL results.

Running everything on two markets is your protection against the classic backtesting sin of cherry-picking: a result that appears on one market and vanishes on the other is telling you something important.

## 2. Environment Design: Choose Your Upgrades

Starting from `HistoricalTradingEnv`, implement **at least two** of the following upgrades. This is the creative core of the project — choose the two you find most interesting, and justify your choices in the report.

**A. Richer state features.** Add 2–3 informative features beyond the raw return window: rolling volatility (std of the last $k$ returns), the price's distance from its own moving average, a momentum indicator over a longer horizon, etc. **Lookahead warning:** every feature at step $t$ must be computable from returns up to and including $t$ — a rolling mean that includes $r_{t+1}$ will silently fake profitability. Re-read the Week 6 golden rule before coding this.

**B. Position sizing.** Replace the 3-action space with 5 actions: full long, half long, flat, half short, full short (positions $+1, +0.5, 0, -0.5, -1$). The transaction-cost term already handles fractional position changes. Does the agent use the intermediate sizes?

**C. Risk-aware reward.** Shape the reward to penalise risk, not just reward profit: subtract a small penalty proportional to recent volatility of the agent's own P&L, or penalise new drawdown lows. Apply the Week 4 test: "if the agent maximised this reward while doing nothing I intended, what would that look like?"

**D. Realistic costs.** Make costs proportional to position change *and* add a per-trade minimum or a spread component. Compare agent behaviour under your cost model vs. the flat Week 4 cost.

**E. Your own idea.** Anything of comparable scope — clear it with your mentor first (one message is enough).

Keep the rest of the environment recognisable: discrete actions, per-step P&L-based reward, 100-step episodes from random windows. The point is controlled improvement, not a rewrite.

## 3. Experiment Protocol

Non-negotiable, all straight from Weeks 5–6:

- **Chronological 80/20 train/test split** per market. All design and tuning decisions use training data only; the test set is touched only for final numbers. If you catch yourself re-running on the test set until you like the result, that is data snooping — stop and say so in the report.
- **At least 3 training runs** (fresh seeds) per configuration. Report mean ± spread across runs, never a single run.
- **Four strategies compared on the test set** of each market:
    1. your improved agent,

2. a *baseline agent* — PPO on the plain Week 6 environment, same budget (this isolates what your upgrades contributed),
3. the random agent,
4. buy-and-hold.

- **Training budget:** 100 000–200 000 timesteps per run is plenty. Do not burn your week on GPU-scale training; burn it on analysis.

### 4. Results to Produce

For each market:

- A **results table**: final mean cumulative P&L, Sharpe ratio, and max drawdown on the test set, for all four strategies, as mean ± spread over your 3+ runs (baselines only need one entry each — they have no seeds worth varying, though random deserves a few repeats).
- A **cumulative P&L plot** on the test set (your `plot_pnl`, extended to four curves).
- At least one **train-vs-test comparison** for your improved agent — your overfitting evidence.

## The Report

The written report is a first-class deliverable, not an afterthought — but it should be **short**: 4–6 pages including plots (or ∼1 000–1 500 words in Markdown with figures). Aim for the tone of a good lab notebook: precise, plain, honest. Required sections:

1. **Introduction** (half page): the question your project asks, in your own words.
2. **Data** (half page): markets chosen and why; the summary-statistics table; anything odd you noticed.
3. **Environment design** (1 page): which upgrades you chose, *why*, and what behaviour you expected each to produce. This section carries the most weight — it shows your judgement.
4. **Experimental setup** (half page): split, seeds, budget, baselines. A reader should be able to reproduce your experiment from this section alone.
5. **Results** (1–2 pages): the tables and plots, described factually. Separate what the numbers *are* from what you *think* they mean.
6. **Discussion** (1 page): Did the upgrades help over the baseline agent? Did anything beat buy-and-hold, and do you believe it? What surprised you? State your single most defensible conclusion in one sentence, in bold.
7. **Limitations and next steps** (half page): what your backtest still ignores (slippage, regime change, one asset class...) and the first thing you would try with two more weeks.

A report that says "none of my agents reliably beat the baselines, and here is the evidence" — with clean methodology behind it — is worth more than one claiming profits from a single lucky seed. You know enough by now to tell which one you are writing.

## Suggested Timeline

- **Week 7, days 1–2:** choose markets, download data, report statistics; decide your two upgrades and sketch expected behaviour *before* coding.
- **Week 7, days 3–5:** implement the upgraded environment; sanity-check it (`check_env`, random episodes, finite rewards — the usual ritual); first full train/evaluate run on one market.
- **Week 7, days 6–7:** baseline-agent runs; start the multi-seed grid. Checkpoint with your mentor: show one results table row.
- **Week 8, days 1–3:** finish all runs on both markets; make final tables and plots.
- **Week 8, days 4–6:** write the report. Budget real time for this — writing is where you find out what you actually learned.
- **Week 8, day 7:** final presentation to your mentor (∼10 minutes): walk through the report, defend your design choices, answer questions.

## Deliverables

- **Code:** your upgraded environment, training/evaluation scripts, and the metrics functions — runnable top to bottom by your mentor.
- **Saved models:** at least one trained model per configuration (`model.save`).
- **Results:** tables and plots as specified above.
- **Report:** PDF or Markdown, 4–6 pages, structure above.
- **Presentation:** 10 minutes with your mentor in the final week.

## What a Strong Project Looks Like

Your mentor will be looking for, roughly in order of importance:

1. **Evaluation discipline.** Clean chronological split, no test-set tuning, multiple seeds, all four strategies, both markets. This is the habit the whole course was building; it weighs the most.
2. **Design judgement.** Upgrades chosen with a stated rationale and an expected behaviour — even better if you then check whether the behaviour actually appeared.
3. **Honest interpretation.** Conclusions sized to the evidence. Noise called noise. Suspicion applied to your own positive results.
4. **Clarity of the report.** A reader who took this course could reproduce your study and would trust your numbers.
5. **Working code.** It runs, it is readable, and the sanity checks are in it.

Nowhere on that list is "the agent made money."

## Common Pitfalls

**Lookahead bias in new features.** The most likely bug in upgrade A. Test: compute your feature at step $t$, then check that changing $r_{t+1}$ in the data does not change it.

**Tuning on the test set.** Every peek at test results that feeds back into a design decision quietly turns your test set into a second training set. Decide, train, evaluate once, report.

**One-seed conclusions.** You saw in Week 6 how much test P&L moves between runs. Any claim not backed by 3+ seeds is an anecdote.

**Scope creep.** Two upgrades done carefully beat five done hastily. If you are behind by the end of Week 7, cut an upgrade, not the evaluation protocol.

**Writing the report last-minute.** The report is half the project. Start drafting the Data and Environment sections during Week 7 — they do not depend on final results.

Good luck — and remember the deepest lesson of the course: the environment, the data, and the evaluation decide what is possible. The algorithm was never the hard part.